

THE ML / DL HANDBOOK · COMPANION SERIES, VOLUME VII

Attention

Every Word, All at Once — Why Recurrence Had to Go

The bottleneck

Query, key, value

Scaled dot-product

Multi-head

Positional encoding

Chapter 20 · From the recurrent cell to the attention matrix

01

What recurrence actually costs

Four separate failures, only one of which the LSTM fixed

02

The first fix: Bahdanau attention

Keep the recurrence, but let the decoder look back at every state

03

Self-attention, from the ground up

Query, key and value — three vectors per word, three matrices per layer

04

The formula, opened up

Score, scale, normalise, mix — with real numbers at every step

05

Multi-head, position and masking

The three things a bare attention layer still needs

06

What it fixes, and what it costs

The scorecard, the quadratic bill, and where an RNN still wins

Recurrence fails in four separate ways

Volume VI fixed exactly one of them. It is worth being precise about which, because the other three are what attention was invented for.

1 • The gradient vanishes

The signal is multiplied by a weight matrix once per step, so it decays exponentially with distance.

Fixed by the LSTM

2 • One vector holds everything

An encoder compresses the whole source sentence into a single fixed-size context vector before the decoder sees a word of it.

Still broken

3 • The path is long

Word 1 reaches word 30 only by being copied and rewritten twenty-nine times. Nothing survives that intact.

Still broken

4 • Nothing runs in parallel

Step t needs h_{t-1} . The GPU that could do the whole sentence at once waits instead.

Still broken

THE HONEST SUMMARY OF VOLUME VI

The conveyor-belt cell state gave the gradient a road home — that is a real, permanent contribution. But the LSTM is still a loop, it still reads one word at a time, it still hands the next step a single summary vector, and it still has to walk the whole distance between two related words. Gating solved the arithmetic; it did not solve the architecture.

FIGURE 20.1

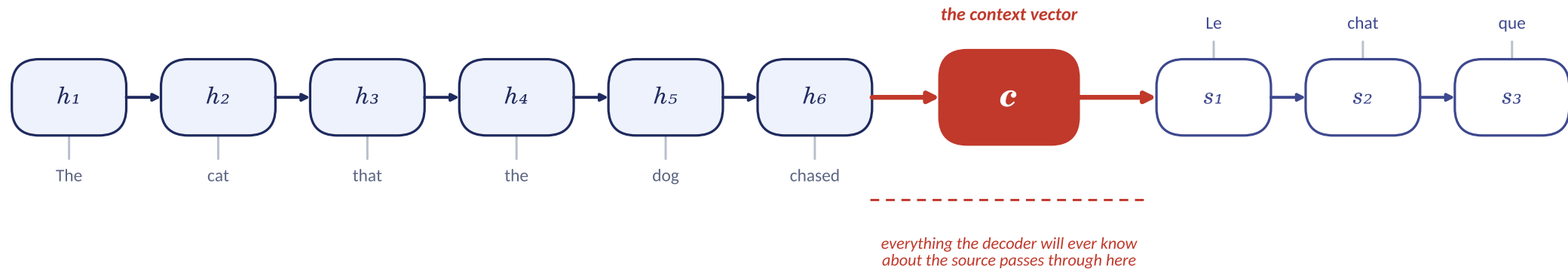
One vector for the whole sentence

The sequence-to-sequence encoder-decoder of 2014: the encoder reads, the decoder writes, and everything in between passes through c .

One fixed-size vector must carry the entire sentence.

ENCODER — reads one word at a time

DECODER — writes the translation from c alone



Add more words and the vector does not get any bigger — the earliest words are simply overwritten.

Figure 20.1: The context vector has the same size for a six-word sentence and a sixty-word one. Translation quality was measured falling off a cliff past roughly thirty tokens — not because the model could not translate, but because the sentence no longer fit.

Distance is the enemy

Every problem with recurrence can be restated as one sentence: the number of operations between two related words grows with how far apart they are.

THE SENTENCE EVERYONE USES

*“The **animal** didn’t cross the street because **it** was too tired.”*

To resolve *it*, the model must reach back seven words. In a recurrent net that means seven multiplications by the same matrix, seven squashing non-linearities, and seven opportunities for the signal to be overwritten by whatever came next.

WHAT THAT DOES TO THE GRADIENT

$\partial h_t / \partial h_k$ is a product of $(t - k)$ Jacobians. If their spectral radius is below 1 the gradient vanishes; above 1 it explodes. There is no setting that is stable at every distance.

THE COST OF EVERY MECHANISM, SIDE BY SIDE

LAYER TYPE	SEQUENTIAL OPS	MAX PATH LENGTH	WORK PER LAYER
Recurrent	$O(n)$	$O(n)$	$O(n \cdot d^2)$
Convolutional	$O(1)$	$O(\log_k n)$	$O(k \cdot n \cdot d^2)$
Self-attention	$O(1)$	$O(1)$	$O(n^2 \cdot d)$

WHY THE MIDDLE COLUMN IS THE ONE TO WATCH

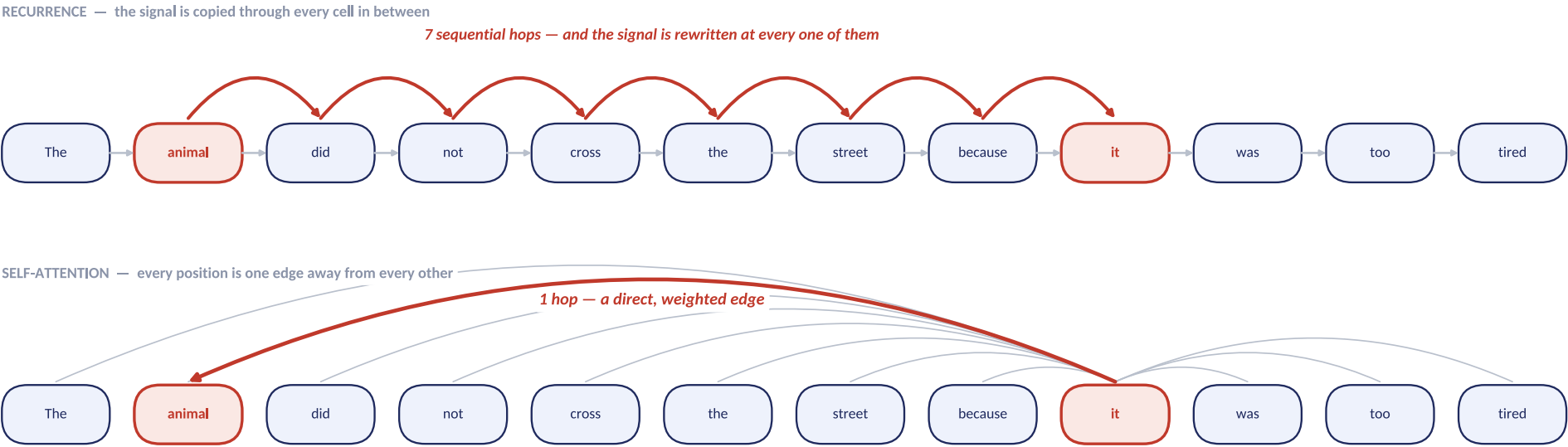
Maximum path length is the distance a gradient has to travel between any two positions. Attention is the only row where that number does not grow with the sentence. Everything else in this chapter follows from that single fact.

FIGURE 20.2

Seven hops, or one

The same sentence, under two mechanisms.

How far must the word “animal” travel to reach the word “it”?



Gradient decay is a function of path length. Attention makes that length 1 for every pair, at every distance.

Figure 20.2: The recurrent path is not merely long — it is lossy at every hop, because each step rewrites the state rather than passing it through. Attention’s edge is direct, weighted, and the same length no matter how far apart the two words sit.

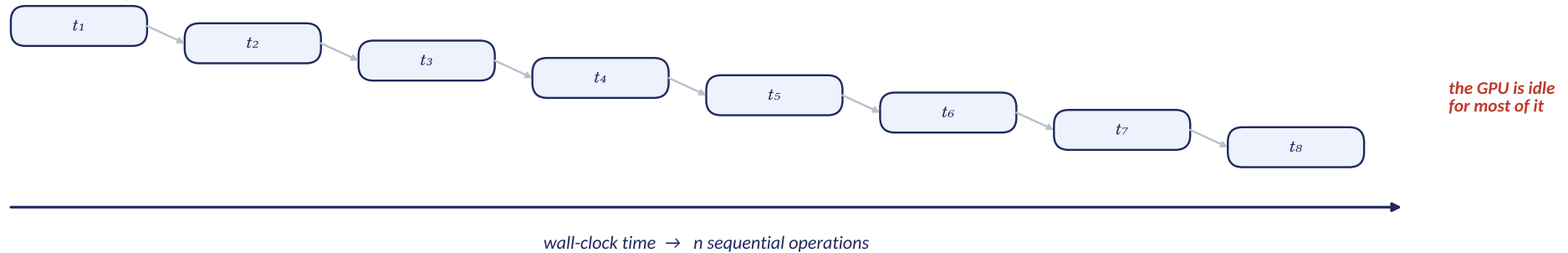
FIGURE 20.3

The constraint that costs the most money

Long before quality, recurrence is a throughput problem.

The recurrence is not just a modelling choice — it is a scheduling constraint.

RNN / LSTM — step t cannot start until step $t-1$ has finished



SELF-ATTENTION — every position is computed inside one matrix multiply

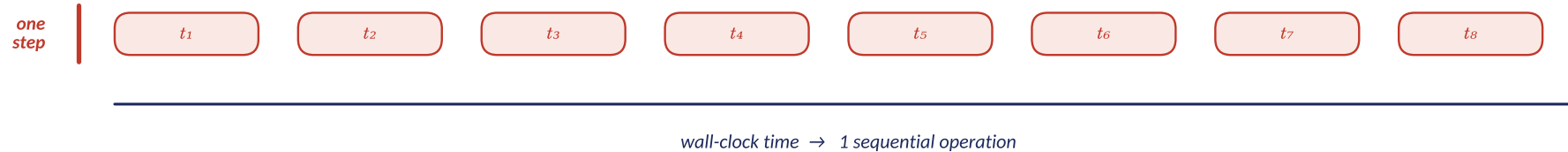


Figure 20.3: A GPU with tens of thousands of cores is asked to compute one hidden state at a time. Removing the recurrence turned a sequential $O(n)$ schedule into a single matrix multiply — and that, more than any accuracy number, is why the field moved.

FROM RECURRENCE TO ATTENTION

**Stop passing the message along.
Let everyone read it directly.**

If the trouble is that information has to travel, the fix is to stop making it travel. Give every position direct access to every other position, and let the model learn how much of each to take.

Bahdanau: keep the states, and look back

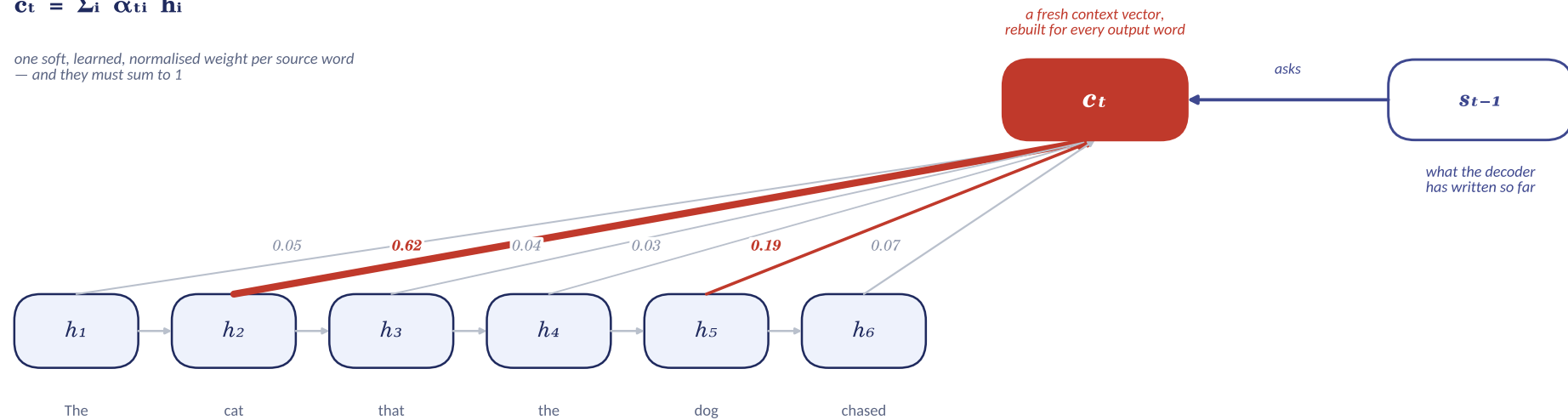
Bahdanau, Cho & Bengio (2014) left the recurrence exactly as it was, and changed only what the decoder is allowed to see.

The first fix: keep the recurrence, but stop throwing the states away.

$$\alpha_{ti} = \text{softmax}(\text{score}(s_{t-1}, h_i))$$

$$c_t = \sum_i \alpha_{ti} h_i$$

one soft, learned, normalised weight per source word
— and they must sum to 1



ENCODER STATES — all of them kept, none discarded

The bottleneck is gone. The recurrence — and everything wrong with it — is still there.

Figure 20.4: Instead of one context vector for the whole translation, the decoder builds a fresh one at every output step — a weighted average of all the encoder states, with the weights produced by a small learned scoring network.

Attention was the useful half all along

Bahdanau attention was proposed as a patch to a recurrent model. Within three years it was clear the patch was doing most of the work.

The bottleneck disappeared

BLEU stopped collapsing on long sentences. Whatever the decoder needed, it could now fetch directly instead of hoping the encoder had kept it.

The weights were interpretable

Plotting α produced a soft word-alignment matrix that matched human intuition — the model had learned alignment without ever being shown one.

The path length went to 1

Every encoder state is one weighted edge from every decoder step. That is the property self-attention would later generalise to every pair inside a single sequence.

BUT THREE OF THE FOUR PROBLEMS SURVIVED

The encoder was still a recurrent loop, so training was still sequential, the encoder's own long-range dependencies were still resolved by walking the chain, and the model still cost $O(n)$ sequential steps. Attention had been added *alongside* recurrence. The obvious question — what happens if you remove the recurrence and keep only the attention — took three years to be asked in print.

VASWANI ET AL., 2017

Attention is all you need.

The title was a claim about ablation, not a slogan: take the encoder-decoder of 2014, delete the recurrent cells, keep the attention, and the model does not merely survive — it trains an order of magnitude faster and scores higher.

Self-attention in one sentence

Every position produces a new representation of itself by taking a weighted average of every position in the sequence — including itself — where it chooses the weights.

1

Compare

Every position scores every other position: how relevant is that word to me, right now?



2

Normalise

Softmax turns each row of scores into a distribution — n weights that sum to 1, one per position.



3

Mix

Take that weighted average of the other positions' vectors. That average is the position's new representation.

WHY THIS IS CALLED "SELF"

In Bahdanau attention the queries came from the decoder and the keys from the encoder — two different sequences. Here the queries, keys and values all come from the *same* sequence. The sentence attends to itself.

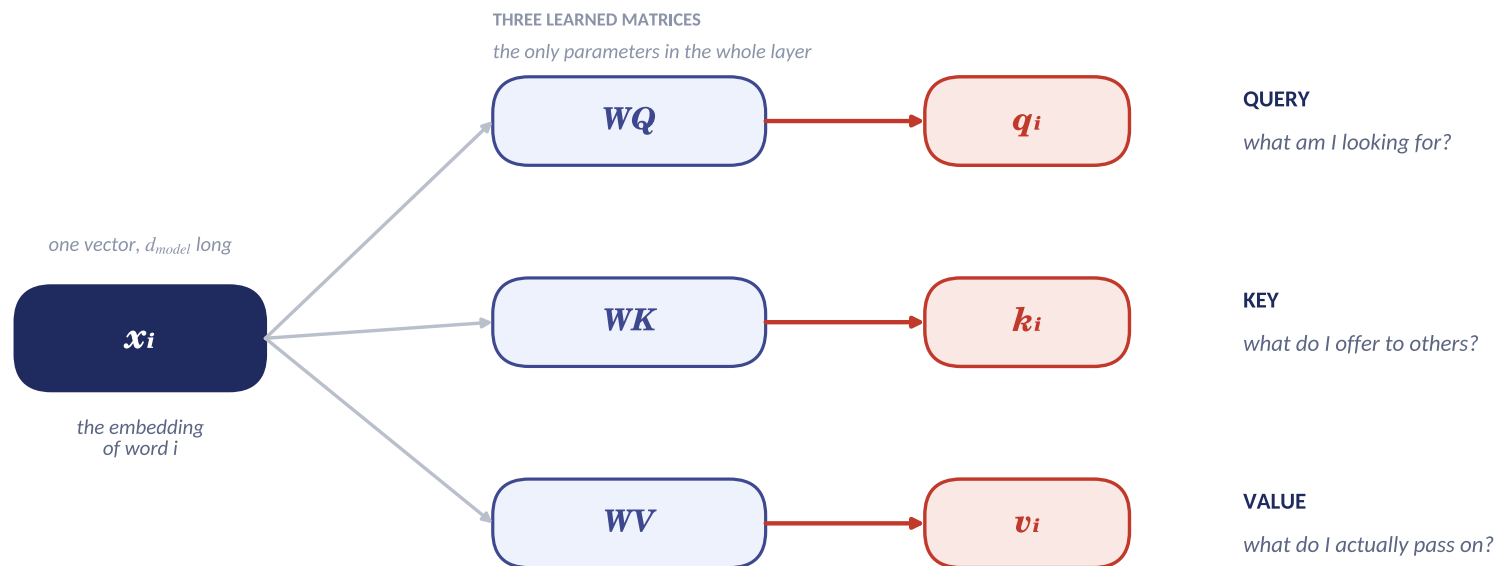
WHAT HAS BEEN REMOVED

There is no hidden state, no loop, no left-to-right order, and no dependence of position t on position $t-1$. The layer is a function of the whole sequence to the whole sequence, computed in one shot.

Query, key, value

The one piece of vocabulary that stops self-attention from feeling arbitrary. It is borrowed, deliberately, from database retrieval.

Every word plays three different roles — so it gets three different vectors.



THE LIBRARY ANALOGY

You walk in with a question (query). Every book has a spine label (key). You compare your question to every spine, then read the books in proportion (value).

Figure 20.5: The same embedding is projected three times, by three different learned matrices, because a word does the three jobs differently: what it is looking for, what it advertises, and what it hands over.

What each matrix is for

Three matrices. That is the entire parameter budget of a single-head self-attention layer — there is nothing else to learn.

 W_Q $d_{\text{model}} \times d_k$

Turns a word into a question. The query vector says what this position wants to find elsewhere in the sentence — “I am a pronoun, where is my noun?” Only the query’s owner ever uses it.

 W_K $d_{\text{model}} \times d_k$

Turns a word into a label. The key is what this position offers to everyone else’s queries — “I am an animate noun.” Queries and keys must land in the same space, because they are about to be dotted together.

 W_V $d_{\text{model}} \times d_v$

Turns a word into content. The value is what actually gets transmitted once a query has decided to attend here. It never takes part in the scoring, which is why it may live in a different-sized space.

**why
three?**

and not one

With a single matrix, “A is relevant to B” would force “B is equally relevant to A” — the score matrix would be symmetric. Separate Q and K make attention **directional**: a pronoun can attend strongly to its noun without the noun attending back. Separating V lets the model decide *what to send* independently of *who to send it to*.

The update, written precisely

One equation, and four operations hidden inside it.

STEP 1 — PROJECT, ONCE, FOR THE WHOLE SEQUENCE

$$Q = X W_Q \quad K = X W_K \quad V = X W_V$$

X is $n \times d_{\text{model}}$, the whole sentence stacked as rows.

STEP 2 — SCORE EVERY ORDERED PAIR

$$S = Q K^T \quad S_{ij} = q_i \cdot k_j$$

An $n \times n$ matrix — and the object that costs quadratic time and memory.

STEP 3 — SCALE, THEN NORMALISE EACH ROW

$$A = \text{softmax}(S / \sqrt{d_k})$$

Row by row, independently: every row of A is a distribution over the n positions.

STEP 4 — MIX THE VALUES

$$Z = A V \quad z_i = \sum_j a_{ij} v_j$$

Row i of Z is a weighted average of every value vector — the new representation of position i .

ALL OF IT AT ONCE

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

READ IT LEFT TO RIGHT

1. Every word asks its question (Q).
2. Every word is compared against every advertised label (K^T).
3. The comparison is divided down to a sane scale ($\sqrt{d_k}$).
4. The comparisons become weights that sum to one (softmax).
5. Those weights average the content (V).

THE SHAPES, SINCE THEY ARE EASY TO LOSE

$$n \times d_{\text{model}} \rightarrow Q, \quad n \times d_k \cdot n \times d_v$$

$$QK^T \ n \times n \cdot A \ n \times n \cdot Z = AV \ n \times d_v$$

Notice that no shape mentions the sequence length twice except the score matrix — and notice that none of them are learned per-position. The same three matrices are applied at every position, exactly as an RNN reuses one cell at every timestep.

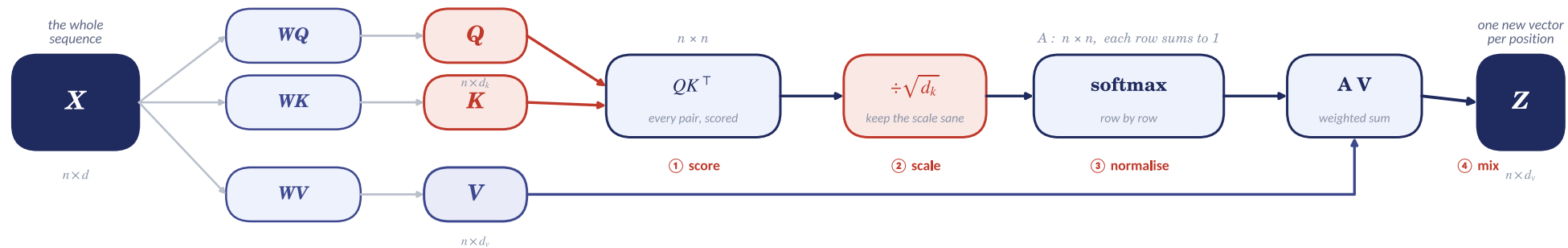
FIGURE 20.6

The attention layer, opened up

Everything a single head does, on one page.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}) \mathbf{V}$$

one layer, five operations, three matrices of parameters — nothing else



NOTICE WHAT IS MISSING

No loop. No hidden state carried from one position to the next. No ordering. Every one of these five operations is a single matrix product over the whole sequence at once — which is exactly why the whole layer fits in one GPU kernel launch.

Figure 20.6: Three projections, one outer product, one scale, one softmax, one weighted sum. Compare this with the LSTM cell of Volume VI: no gate, no carried state, and no arrow that points back to the previous timestep.

Four words, four dimensions, no hand-waving

The smallest example in which the mechanism is still the real mechanism: $n = 4$ tokens, $d_k = 4$.

THE KEYS — WHAT EACH WORD ADVERTISES

$k(\text{the}) = [1 \ 0 \ 0 \ 0]$

$k(\text{cat}) = [0 \ 2 \ 0 \ 0]$

$k(\text{sat}) = [0 \ 0 \ 2 \ 0]$

$k(\text{down}) = [0 \ 0 \ 0 \ 1]$

Read dimension 2 as “I am a noun”, dimension 3 as “I am a verb”. Nothing forces this interpretation — it is what training produces.

THE QUERIES — WHAT EACH WORD IS LOOKING FOR

$q(\text{the}) = [2 \ 0 \ 0 \ 0]$

$q(\text{cat}) = [0 \ 2 \ 0 \ 0]$

$q(\text{sat}) = [0 \ 3 \ 1 \ 0]$

$q(\text{down}) = [0 \ 0 \ 2 \ 1]$

*The verb **sat** puts its weight on the noun dimension: it is looking for a subject. The adverb **down** looks for a verb.*

WHAT WE ARE ABOUT TO COMPUTE

1. $S = QK^T$ — sixteen dot products, one per ordered pair.
2. $S / \sqrt{4} = S / 2$ — the scaling step.
3. softmax along each row — four distributions.
4. $Z = AV$ — each word’s new vector.

THE PREDICTION, BEFORE WE LOOK

Row 3 (*sat*) should put most of its weight on *cat*, because $q(\text{sat})$ and $k(\text{cat})$ both live on dimension 2 and their dot product is $3 \times 2 = 6$ — the largest number anywhere in the matrix.

Row 4 (*down*) should favour *sat*, at $2 \times 2 = 4$.

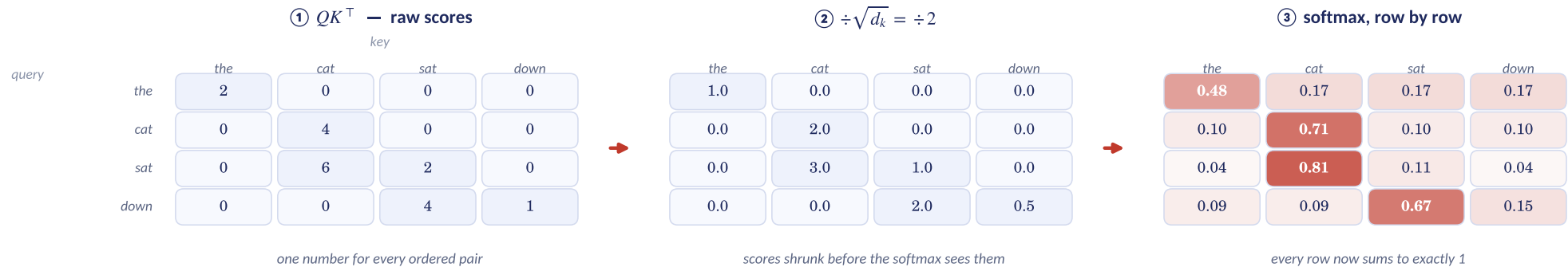
Rows 1 and 2 are near-diagonal: these words are looking for what they themselves are.

FIGURE 20.7

Score, scale, softmax — with numbers

The same three steps as the formula, evaluated.

The same three steps, with real numbers ($n=4$, $d_k=4$)



READING ROW 3

The row for “sat” puts 0.81 of its weight on “cat” and almost nothing anywhere else. The output vector for “sat” is therefore $0.81 \cdot v(\text{cat}) + 0.11 \cdot v(\text{sat}) + 0.04 \cdot v(\text{the}) + 0.04 \cdot v(\text{down})$. Nothing in the layer told it to do that — the three weight matrices were trained until that is what the dot products came out to be.

Figure 20.7: The attention matrix A. Every row sums to 1 by construction, and the largest entry of each row is what that word decided to read. Nothing here is a heuristic — the pattern is the arithmetic of the dot product, and training is what puts the vectors where the dot products come out this way.

Why divide by $\sqrt{d_k}$

The most-skipped line in the paper, and the one that decides whether the layer trains at all.

ASSUME THE COMPONENTS ARE INDEPENDENT, MEAN 0, VARIANCE 1

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i$$

Each term has mean 0 and variance 1.

VARIANCE ADDS OVER INDEPENDENT TERMS

$$\text{Var}(q \cdot k) = d_k \Rightarrow \sigma = \sqrt{d_k}$$

At $d_k = 512$ the typical score is not ± 1 — it is ± 23 .

SO DIVIDE BY EXACTLY THAT

$$S / \sqrt{d_k} \Rightarrow \text{Var} = 1, \text{ at any width}$$

The scaling is not tuned. It is the standard deviation the quantity already has, divided back out.

WHAT GOES WRONG WITHOUT IT

Softmax is exponential. Feed it scores that differ by 20 and it returns something indistinguishable from one-hot: one weight of 1.000, the rest 0.000.

That is not merely a bad average — it is a **dead gradient**. The Jacobian of softmax is $\text{diag}(a) - aa^T$, which is ≈ 0 everywhere when a is one-hot. The layer stops learning where to look, and it stops in the first few steps of training, before the weights mean anything.

WORTH KNOWING — THE SAME IDEA, ELSEWHERE

This is the same reasoning behind Xavier and He initialisation: keep the variance of an activation at 1 as it passes through a layer, by dividing by the square root of the fan-in. Attention has a fan-in of d_k , so it divides by $\sqrt{d_k}$. Once you see it as a variance-control device rather than a magic constant, the whole line stops being mysterious.

FIGURE 20.8

What the scale factor actually buys

Left: the problem. Middle: what softmax does about it. Right: the fix.

$\sqrt{d_k}$ is not a hyper-parameter — it is the standard deviation the dot product already has.

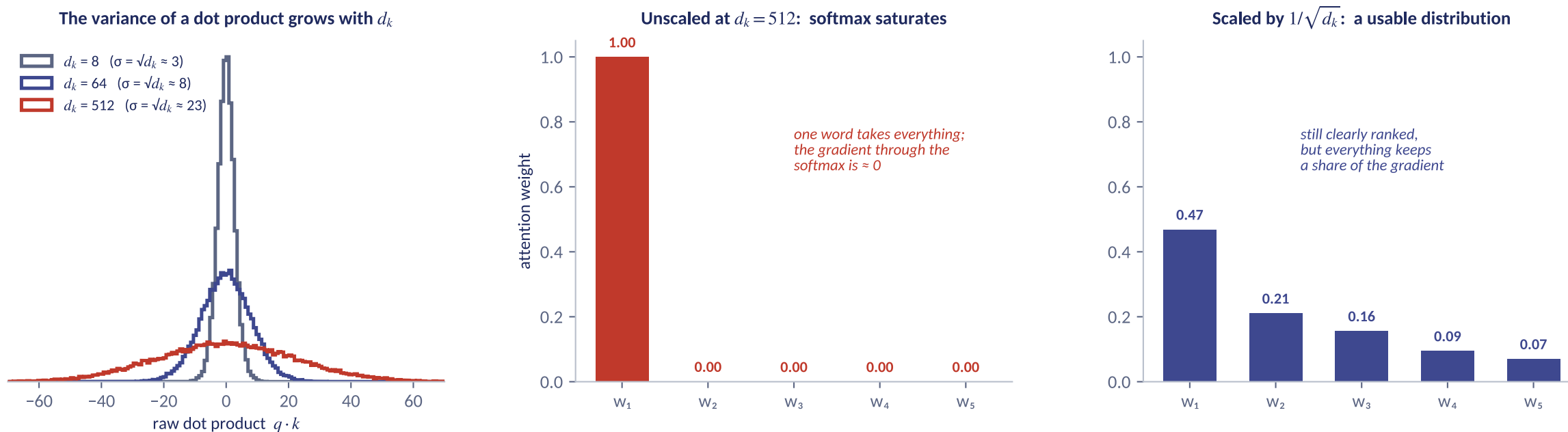


Figure 20.8: The dot product of two random d -dimensional vectors has standard deviation $\sqrt{d}$. Without the correction, the softmax at realistic widths is saturated from initialisation and no gradient reaches the query and key matrices.

FROM ONE HEAD TO MANY

One head can hold one opinion at a time.

A single softmax row is a single distribution. If a verb needs to find its subject *and* its object *and* the word before it, one set of weights cannot express all three at once — averaging them produces something that is none of them.

Run the whole thing h times, narrower

Not h times the cost — the same cost, cut into h independent slices.

Split

$d_{\text{model}} = 512$ is divided into 8 heads of $d_k = 64$. Each head gets its own W_Q, W_K, W_V , each 512×64 .

Attend, independently

Each head runs the full scaled dot-product attention over the whole sequence and produces its own $n \times 64$ output. The heads never see each other while doing it.

Concatenate

The eight $n \times 64$ outputs are glued side by side into one $n \times 512$ matrix — back to the width the layer started at.

Mix

A final W_O (512×512) lets the heads' findings combine. Without it the heads would stay in eight sealed sub-spaces forever.

THE ARITHMETIC THAT MAKES IT FREE

One head at width 512: three matrices of $512 \times 512 = 786k$ parameters.

Eight heads at width 64: $8 \times 3 \times (512 \times 64) = 786k$ parameters.

Identical. The score matrices are eight $n \times n$ instead of one, and the multiplies are eight times narrower. The heads are a partition of the width, not an addition to it.

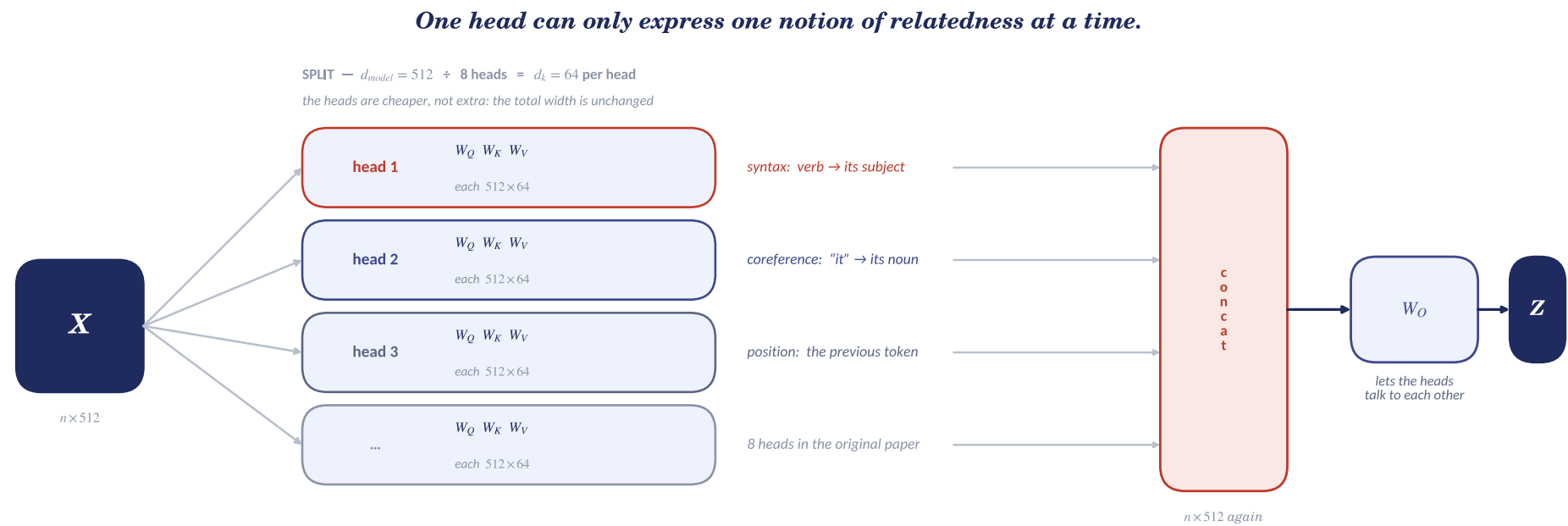
WHAT THE HEADS BECOME

Nothing assigns a head a job. They diverge because two identical heads would be wasted capacity, and gradient descent does not waste capacity. Probing trained models finds heads that track syntactic dependencies, heads that resolve coreference, heads that simply look one token back — and a good many heads that can be pruned after training with no measurable loss.

FIGURE 20.9

Eight views of the same sentence

Multi-head attention, split to concatenation.



Eight heads at width 64 cost the same as one head at width 512 — and buy eight independent views of the sentence.

The heads are never told what to specialise in. They diverge because identical heads would waste capacity.

Figure 20.9: The width is partitioned, not multiplied. Every head sees the whole sequence but only 64 of the 512 dimensions of each word — which is exactly what forces them to specialise.

Every shape in a multi-head layer

n is the sequence length, $d = d_{\text{model}} = 512$, $h = 8$ heads, $d_k = d_v = 64$.

Object	Shape	Learned?	What it is
X	$n \times 512$	no	The input sequence — embeddings plus positional encoding.
$W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$	512×64 , $\times 3$ per head	yes	The three projections belonging to head i . 24 matrices in total across 8 heads.
$Q^{(i)}, K^{(i)}, V^{(i)}$	$n \times 64$	no	Head i 's view of the sequence. Recomputed from X on every forward pass.
$A^{(i)} = \text{softmax}(\cdot)$	$n \times n$	no	Head i 's attention matrix. Eight of these exist at once, per layer — this is where the memory goes.
$\text{head}_i = A^{(i)}V^{(i)}$	$n \times 64$	no	One head's output.
W_O	512×512	yes	Applied after concatenation, so the heads can be combined rather than merely stacked.

Total learned parameters in the layer: $4 \times 512 \times 512 \approx 1.05\text{ M}$ — three projections plus the output mix, and not one of them depends on n . The same layer handles a five-token sentence and a five-thousand-token document.

Attention cannot tell word order

Permute the input rows and the output rows permute with them, unchanged. Self-attention is a set operation — and a sentence is not a set.

THE PROBLEM, STATED EXACTLY

For any permutation matrix P : $\text{Attention}(PX) = P \cdot \text{Attention}(X)$.

The RNN got order for free, because it read left to right. The moment the loop is removed, “*dog bites man*” and “*man bites dog*” become the same input.

THE FIX: PUT THE ORDER IN THE VECTOR, NOT THE MECHANISM

Add a position-dependent vector to every embedding before the first layer. The attention mechanism stays permutation-equivariant; the *input* stops being permutation-invariant. Two occurrences of the same word at different positions now arrive as different vectors.

THE SINUSOIDAL CHOICE

$$\text{PE}_{(\text{pos}, 2i)} = \sin(\text{pos} / 10000^{2i/d})$$

$$\text{PE}_{(\text{pos}, 2i+1)} = \cos(\text{pos} / 10000^{2i/d})$$

WHY SINUSOIDS RATHER THAN JUST THE INTEGER INDEX

- **Bounded.** Every component stays in $[-1, 1]$ regardless of sequence length; a raw index would swamp the embedding it is added to.
- **Multi-scale.** Low dimensions oscillate every few tokens, high dimensions over thousands — so the same vector encodes “next word” and “several paragraphs ago”.
- **Relative position is linear.** $\text{PE}(\text{pos} + k)$ is a fixed linear function of $\text{PE}(\text{pos})$ — a rotation. A head can learn “attend three tokens back” as a single matrix.
- **Extrapolates.** The function is defined at positions longer than anything seen in training. A learned embedding table simply has no row there.

WORTH KNOWING — WHAT IS ACTUALLY USED NOW

Learned absolute embeddings (BERT, GPT-2) work about as well in-distribution and were long the default. Modern long-context models have largely moved to **rotary embeddings (RoPE)**, which rotate Q and K by a position-dependent angle so that the dot product depends only on the *relative* offset — the same intuition as the sinusoids, applied inside the attention score instead of on the input.

FIGURE 20.10

Order, added to the input

The sinusoidal positional encoding, drawn.

Attention is order-blind by construction — so order is added to the input, not to the mechanism.

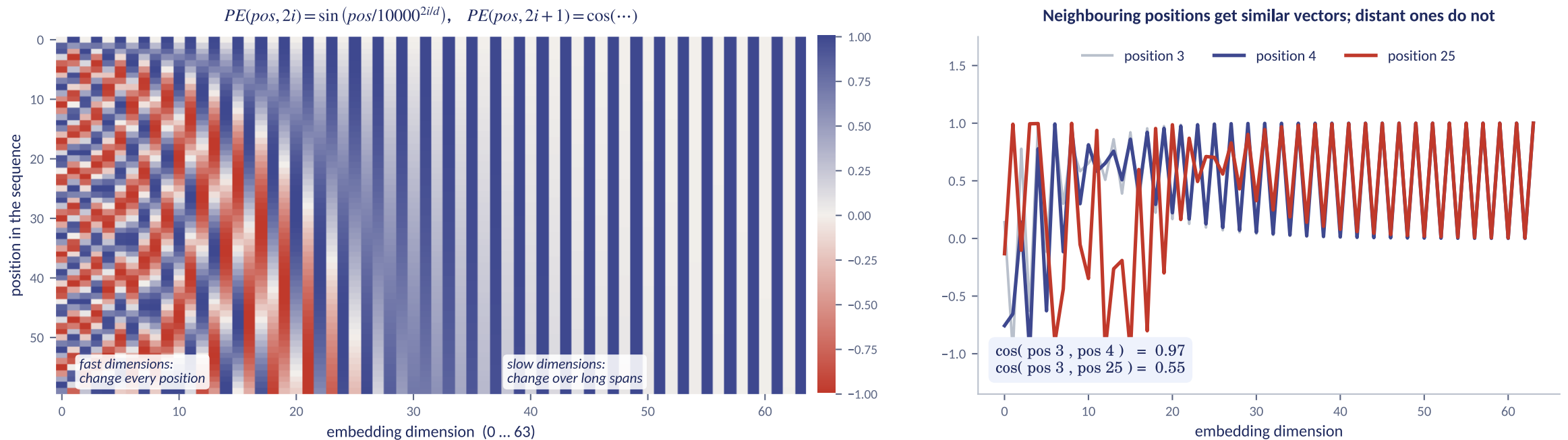


Figure 20.10: Each row is one position's vector. Nearby rows are similar and distant rows are not, so a dot product between two positional encodings is already a rough measure of distance — which is precisely what an attention score can use.

Masking: not everything may be looked at

Two different reasons to forbid an attention edge, both implemented the same way — by setting the score to $-\infty$ before the softmax.

Causal mask

A language model predicting token t may not attend to tokens $t+1$ and beyond — it would be reading the answer. Every position is masked to its right.

GPT-style decoders

Padding mask

Batched sequences are padded to equal length. The pad tokens carry no information and must never receive attention weight from a real token.

Every batched model

No mask

An encoder classifying a complete sentence should see all of it, both directions at once. This is the whole difference between BERT and GPT.

BERT-style encoders

WHY THE MASK IS WHAT MAKES PARALLEL TRAINING POSSIBLE

With a causal mask, one forward pass over an n -token sequence computes the prediction at *every* position simultaneously — position 5 is trained on the prefix of length 5, position 900 on the prefix of length 900, in the same matrix multiply. An RNN language model gets exactly the same n training signals, but has to take n sequential steps to collect them. This is the single largest reason pre-training scaled.

THE IMPLEMENTATION DETAIL THAT BITES

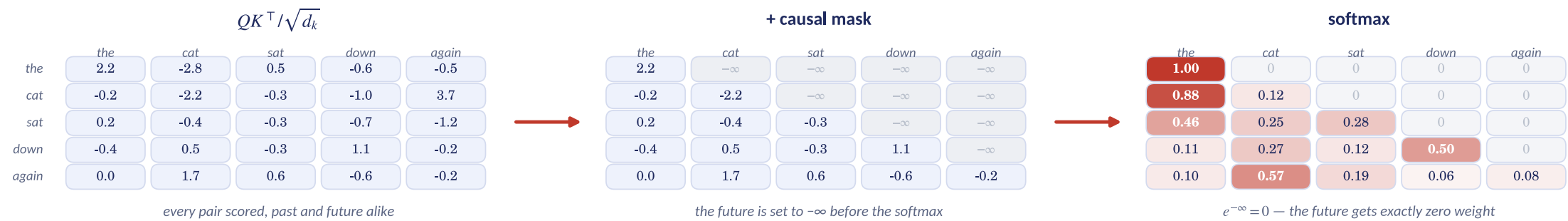
Add $-\infty$ (in practice $-1e9$, or the dtype's minimum) to the masked scores *before* the softmax, never zero the weights after it. Zeroing afterwards leaves the surviving row un-normalised, and a row that no longer sums to 1 quietly rescales every value vector it touches.

FIGURE 20.11

The causal mask, step by step

Scores, masked scores, and the distribution that comes out.

A language model may not look at the answer it is being asked to predict.



WHY $-\infty$ AND NOT ZERO
Zeroing a score would still leave $e^0 = 1$ of weight on the future. Setting it to $-\infty$ before the softmax makes the weight exactly zero, and keeps every row a proper distribution over the visible positions. Encoders (BERT) use no mask at all; decoders (GPT) mask every position to its right.

Figure 20.11: The first row can only see itself, so its weight is 1.00 by necessity. Each later row spreads over a longer visible prefix. The upper triangle is not small — it is exactly zero.

Everything is $O(n^2)$, and that is the whole catch

Self-attention buys constant path length by explicitly materialising every pair. There is no version of it that does not pay for that.

Every pair is scored — so the work grows with the square of the sequence length.

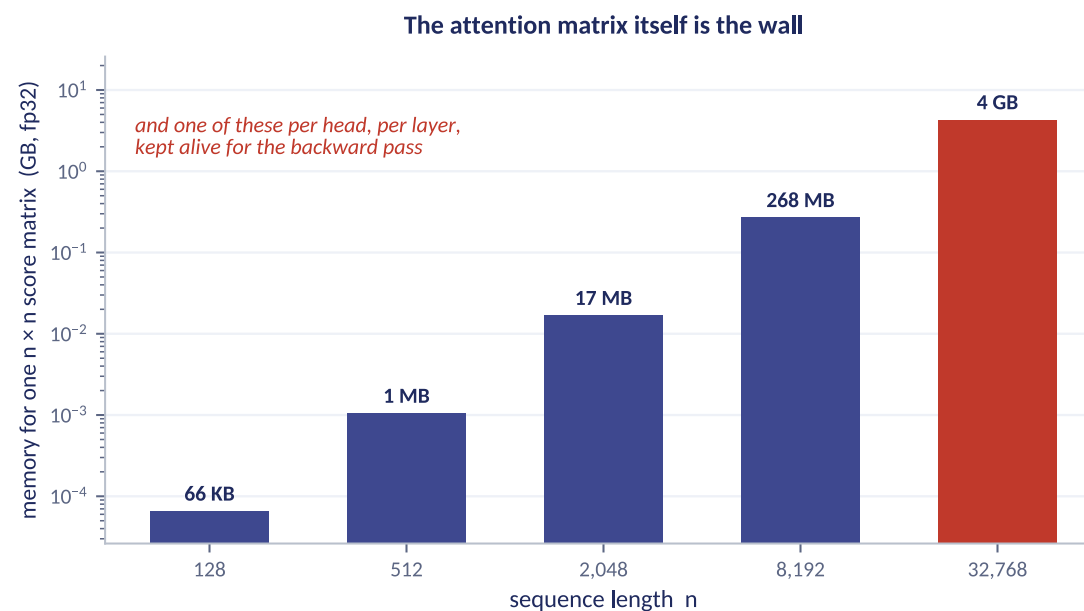
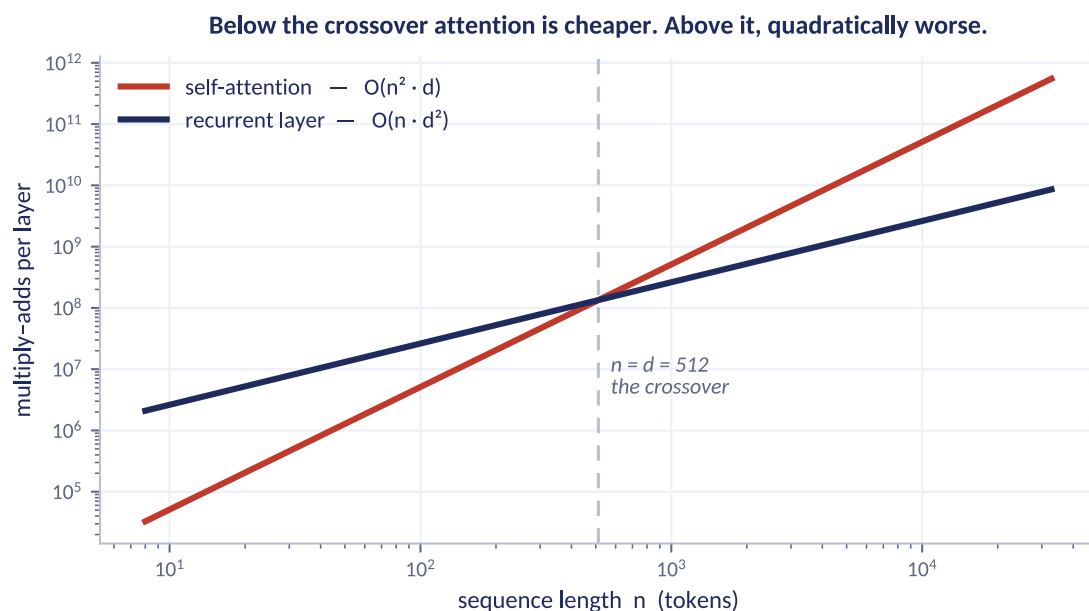


Figure 20.12: At the paper's $d = 512$ and $n \leq 512$, attention is genuinely cheaper than recurrence per layer. At $n = 32,768$ a single fp32 score matrix is 4 GB — per head, per layer — which is why long-context work is dominated by FlashAttention (never materialise the matrix), sparse and sliding-window patterns, and linear-attention approximations.

Every problem from slide 3, answered

This is the slide the whole chapter exists to earn.

The RNN / LSTM problem	What self-attention does about it	Verdict
Vanishing gradient over distance	Position i reads position j through one weighted edge, at any distance. There is no product of Jacobians to decay, because there is no chain — every pair is depth-1.	Solved
One fixed-size context vector	Nothing is ever compressed into a summary. Every position keeps its own vector, and any position may read any other one directly.	Solved
Long path between related words	Maximum path length is $O(1)$ rather than $O(n)$ — the property the whole architecture is built around.	Solved
No parallelism during training	Zero sequential operations. The layer is four matrix multiplies and a softmax over the entire sequence at once, on all positions simultaneously.	Solved
Fixed, opaque mixing	The mixing weights are <i>computed from the content itself</i> at inference time — a different attention matrix for every input, rather than one weight matrix for all of them.	Bonus
— but order came for free	Lost. It has to be added back explicitly through positional encoding, and getting that right at long context is still an active research problem.	New cost
— and cost was linear in n	Now quadratic in time and memory. Below $n \approx d$ attention is still cheaper; well above it, this is the binding constraint.	New cost

What self-attention does not fix

Three claims that sound like they should follow from the last slide, and do not.

It does not *understand*

An attention weight is a normalised dot product, not a claim about meaning. Attention maps are suggestive but are **not** reliable explanations — several results show you can substantially change the weights without changing the prediction. Treat them as a debugging aid, not evidence.

O(1) path $\neq$ perfect memory

A short path makes long-range information *reachable*; it does not make it *salient*. Softmax over thousands of positions spreads thin, and the well-documented “lost in the middle” effect shows long-context models still under-use what sits far from the start and the end of the input.

Attention alone is not a model

Stack pure attention layers and the representation degenerates — outputs converge towards one another with depth. What makes a working Transformer block is attention **plus** a position-wise feed-forward network, residual connections and layer normalisation. That is Volume VIII.

And it is not always the answer

On short sequences with little data, a GRU frequently still wins — it has fewer parameters, a strong sequential inductive bias, and it does not have to learn word order from scratch. Attention is a trade of prior knowledge for capacity, and that trade only pays when there is data to fill the capacity.

NONE OF WHICH IS AN ARGUMENT AGAINST IT

Every one of these is a limit on what attention *is*, not a reason to reach for something else. The mechanism removed the two constraints that had capped sequence models for twenty years — the travelling and the waiting — and the field has spent the years since working on the bill it left behind rather than looking for a replacement.

Where each one still wins

Attention displaced recurrence almost everywhere — but “almost” is doing real work in that sentence.

Recurrence still wins

- **True streaming.** One token in, constant memory, constant work per step. Attention must re-read a growing history.
- **Very long, low-value sequences.** Sensor traces, audio frames — $O(n)$ beats $O(n^2)$ long before the quality difference matters.
- **Small data.** The sequential prior is free supervision when there is not enough data to learn it.
- **Tight edge budgets.** No $n \times n$ buffer to hold.

Attention wins

- **Anything trained at scale.** Parallel training is the difference between a week and a month of GPU time.
- **Long-range dependence.** Coreference, discourse, code — every case where the two related tokens are far apart.
- **Transfer.** Pre-train once on unlabelled text, fine-tune everywhere. The recipe that made the last decade of NLP.
- **Anything but text.** Vision, audio, protein sequences, graphs — the same layer, with a different tokeniser.

IN PRACTICE

If the sequence is short, the data is small, or the setting is genuinely streaming, reach for a GRU first. Everywhere else, and certainly anywhere you intend to fine-tune something pre-trained, the question is not whether to use attention but which attention variant the sequence length forces on you.

Key takeaways

- 1 Recurrence fails in four ways and the LSTM fixed one. Gating rescued the gradient; it left the bottleneck, the long path and the sequential schedule exactly where they were.
- 2 Self-attention replaces travel with direct access. Every position reads every position through one weighted edge, so the maximum path length is $O(1)$ at any distance.
- 3 Three learned matrices — W_Q , W_K , W_V — give each word a question, a label and a payload. Separating them is what makes attention directional rather than symmetric.
- 4 $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$. The $\sqrt{d_k}$ is not a hyper-parameter: it is the standard deviation the dot product already has, divided out to stop the softmax saturating.
- 5 Multi-head is a partition of the width, not a multiplication of it — eight heads at 64 cost what one head at 512 costs, and buy eight independent notions of relatedness.
- 6 Removing the loop removes word order, so position is added to the input; and scoring every pair costs $O(n^2)$ time and memory, which is the constraint every long-context technique since has been trying to get around.

End of Volume VII

Companion to Part VII of the ML / DL Handbook

NEXT

Volume VIII — The Transformer.

The encoder block, residuals and layer norm, the feed-forward network, cross-attention, the decoder, and how the whole stack is trained.

SOURCES

Vaswani et al., “Attention Is All You Need”, NeurIPS 2017 · arXiv:1706.03762

Bahdanau, Cho & Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate”, ICLR 2015 · arXiv:1409.0473

Dong, Cordonnier & Loukas, “Attention Is Not All You Need: Pure Attention Loses Rank Doubly Exponentially with Depth”, ICML 2021

Liu et al., “Lost in the Middle: How Language Models Use Long Contexts”, TACL 2024